

When we create a NumPy array, NumPy stores data in a specific format for **speed** and **memory optimization**.

Example:

```
import numpy as np
```

```
arr = np.array([1, 2, 3])  
print(arr.dtype)
```

Output:

```
int64
```

Here int64 belongs to integer type (i).

NumPy Data Type Codes Explained

Code	Meaning	Example
i	Integer	1, 2, -5
b	Boolean	True, False
u	Unsigned Integer	Only positive numbers
f	Float	1.5, 3.14
c	Complex Float	2+3j
m	Timedelta	Time difference
M	Datetime	Date & time
O	Object	Mixed data
S	Byte String	b'Hello'
U	Unicode String	"Hello"
V	Void / Raw Memory Fixed binary data	

1. Integer (i)

Stores whole numbers.

```
import numpy as np
```

```
arr = np.array([10, 20, 30], dtype='i')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[10 20 30]
int32
```

Real-time use:

- Age
 - Quantity
 - Product count
 - Sales units
-

2. Boolean (b)

Stores True or False.

```
arr = np.array([True, False, True], dtype='b')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[ True False  True]
bool
```

Real-time use:

- Login success/failure
 - Paid/Unpaid
 - Active/Inactive
-

3. Unsigned Integer (u)

Stores only positive integers.

No negative values allowed.

```
arr = np.array([1, 2, 255], dtype='uint8')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[ 1  2 255]
uint8
```

Why use this?

Because no negative values → saves memory.

Real-time use:

- Image pixel values (0–255)
 - RGB colors
-

4. Float (f)

Stores decimal numbers.

```
arr = np.array([1.5, 2.7, 3.9], dtype='f')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[1.5 2.7 3.9]
float32
```

Real-time use:

- Salary
 - Temperature
 - Percentage
 - Scientific calculations
-

5. Complex Float (c)

Used in advanced mathematics.

A complex number has:

- Real part
- Imaginary part (j)

```
arr = np.array([2+3j, 4+5j], dtype='c')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[2.+3.j 4.+5.j]
```

```
complex64
```

Real-time use:

- Electrical engineering
 - Signal processing
 - AI research
 - Physics
-

6. Timedelta (m)

Represents duration or time difference.

```
arr = np.array([5, 10, 15], dtype='timedelta64[D]')
```

```
print(arr)
```

Output:

```
[ 5 10 15]
```

Means:

- 5 days
- 10 days
- 15 days

Real-time use:

- Delivery days
 - Employee working duration
 - Project duration
-

7. Datetime (M)

Stores dates and times.

```
arr = np.array(['2026-05-18', '2026-06-01'], dtype='datetime64')
```

```
print(arr)
```

```
print(arr.dtype)
```

Output:

```
['2026-05-18' '2026-06-01']
```

```
datetime64[D]
```

Real-time use:

- Order dates
 - Attendance dates
 - Sales timestamps
-

8. Object (O)

Can store mixed data types.

```
arr = np.array([1, "hello", True], dtype='O')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[1 'hello' True]
object
```

Why avoid this sometimes?

Object arrays are slower because NumPy cannot optimize them well.

Real-time use:

- Mixed columns
 - Complex custom objects
-

9. String (S)

Stores byte strings.

```
arr = np.array([b'Apple', b'Banana'], dtype='S')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[b'Apple' b'Banana']
|S6
```

Real-time use:

- Binary text data
 - Encoded files
-

10. Unicode String (U)

Stores normal text strings.

```
arr = np.array(['Apple', 'Banana'], dtype='U')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
['Apple' 'Banana']
<U6
```

Real-time use:

- Customer names
 - Product names
 - Cities
-

11. Void (V)

Stores raw binary/fixed memory data.

Rarely used in normal analytics.

```
arr = np.array([(1, 2)], dtype='V8')
```

```
print(arr)
print(arr.dtype)
```

Output:

```
[b'\x01\x00\x00\x00\x02\x00\x00\x00']
|V8
```

Real-time use:

- Low-level systems programming
 - Binary file processing
-

Important Concept: Why NumPy Data Types Matter

NumPy is fast because:

- It stores same-type data together
- Uses fixed memory size
- Performs vectorized operations

Example:

```
arr = np.array([1,2,3], dtype='int8')
```

int8 uses only 1 byte per number.

But:

```
arr = np.array([1,2,3], dtype='int64')
```

Uses 8 bytes per number.

So choosing correct datatype helps:

- Reduce memory
- Improve speed
- Handle large datasets

Real-Time Data Analyst Example

Imagine sales data:

```
sales = np.array([1000, 2000, 3000], dtype='i')
profit = np.array([10.5, 20.8, 15.2], dtype='f')
is_target_achieved = np.array([True, False, True], dtype='b')
dates = np.array(['2026-05-01', '2026-05-02'], dtype='datetime64')
```

Here:

- Sales → Integer
- Profit → Float
- Target status → Boolean
- Date → Datetime

This is how NumPy is used in real analytics projects.